```
  "properties": [],
  "constraints": [ "sensor == temperature" ]
}
```

Publish the deployment policy:

```
hzn exchange deployment addpolicy -f deployment.policy.json
temp-sensor-policy
hzn exchange deployment listpolicy
```

Write the node policy: Save as *node.policy.json* on the edge device:

```
{
  "properties": [ { "name": "sensor", "value": "temperature"
} ],
  "constraints": []
}
```

Register the node with the policy: The all-in-one node is already running the sample 'hello-world' pattern, so unregister it first.

```
hzn unregister -f
hzn register --policy node.policy.json
```

Wait for the agreement: Within about a minute, the agbot notices that the node's properties satisfy the policy's constraint, proposes an agreement, and the agent pulls and starts the container.

```
watch hzn agreement list
```

The same policy now applies to every future node. Any new device registered with *sensor=temperature* receives *temp-sensor* automatically, with no further commands on the hub.

Interaction with the service

Once the service runs, you interact with it and with the fleet through the *hzn* CLI on the edge node and on the hub.

View the service output: The agent forwards container logs, so you see the simulated readings:

```
hzn service log -f temp-sensor

# 2026-10-05T09:14:05Z node=node1 temperature=24.7C
# 2026-10-05T09:14:10Z node=node1 temperature=21.3C
```

Common commands are:

Task	Command
Show this node's state and policy	*hzn node list and hzn policy list*
See active agreements	*hzn agreement list*
See running services	*hzn service list or docker ps*
Read the agent's event log	*hzn eventlog list*
List all nodes in the organisation (hub)	*hzn exchange node list*
Check why a policy does or does not match	*hzn deploycheck all -b myorg/ temp-sensor-policy*
Remove the workload and leave the hub	*hzn unregister -f*

Update the service: To roll out version 1.0.1, build and push the new image, change *version* in *service.definition.json*, publish it, and add *{ "version": "1.0.1" }* to the policy's *serviceVersions*. Every matching node upgrades itself. Listing the older version as well allows the agent to roll back if the new one fails.

Change behaviour with properties: Removing the property with hzn policy update on a node makes the agbot cancel the agreement and stop the container on that node only. This is how a single device is taken out of a rollout without touching the others.

Open Horizon turns the hard problem of running IoT software on thousands of unattended devices into a declarative one: you publish a service and a policy once, and autonomous agents place, run, update and heal the workloads on every matching device. In this walkthrough, one script installed a complete management hub, agent and CLI on an Ubuntu machine. A simulated temperature-sensor container was then built, signed, published and deployed through a policy, and monitored with the *hzn* CLI.

Because it is open source and governed by LF Edge, Open Horizon offers enterprise-grade capabilities without vendor lock-in. These include policy-based placement, separate machine learning model delivery, edge-cluster support, signed services and zero-touch onboarding. Natural next steps are to add a Raspberry Pi as a second node, deliver a machine learning model through the Model Management System, and connect the sensor output to a message broker such as MQTT or Kafka for cloud analytics. END

References

- Open Horizon Quick Start
- Prerequisites for installing the management hub
- Creating your own 'hello world' edge service
- *deploy-mgmt-hub.sh* script (open-horizon/devops)
- Open Horizon examples repository

By: Dr R. Anand

The author is a professor in the Department of Computer Science and Engineering at Easwari Engineering College, Chennai, Tamil Nadu.